



TAAL Digital Stethoscope SDK

Integration Guide for Android

taal-core + taal-ui-kit

Version: 2.0.0

Date: March 2026

Version History

Version	Date	Description
2.0.0	March 2026	Major release. New taal-core.aar and taal-ui-kit.aar. Package rename from in.museinc.android.surr_core to com.musediagnostics.taal. Pre-amp extended to 0-30 dB. Dual file output (raw + filtered). BPM calculation. setPreFilter added to TaalPlayer. Max recording time 300 seconds. Portrait-only orientation enforced.
1.0.2	Oct 29, 2021	UI-Kit TAAL Player loop fix. UI changes. Background operation stopped on screen exit.
1.0.1	Oct 04, 2021	Added Custom pre filter. Added detailed error messages. Bug/issue fixing.
1.0.0	Sep 08, 2021	First edition (surr-core + surr-ui-kit).

1. Introduction

The MUSE Diagnostics TAAL SDK for Android provides complete functionality to record, process, and play back audio from the TAAL digital stethoscope device. It is designed for Android developers integrating stethoscope functionality into clinical applications.

The SDK ships as two .aar files:

- taal-core.aar — Pure audio engine with no UI dependency
- taal-ui-kit.aar — Pre-built recording and playback UI (depends on taal-core)

2. SDK Types

2.1 taal-core

Provides core recording and playback functionality with no pre-built UI. Use this when you want full control over your own interface.

Package: com.musediagnostics.taal

2.2 taal-ui-kit

Provides pre-built Activities and Fragments for recording and playback with a complete waveform UI. Depends on taal-core.

Package: com.musediagnostics.taal.uikit

3. Migration from surr-core / surr-ui-kit

IMPORTANT: This section is required reading if you previously used surr-core.aar or surr-ui.aar.

3.1 AAR File Rename

Old File	New File
surr-core.aar	taal-core.aar
surr-UI-Kit.aar	taal-ui-kit.aar

3.2 Package Name Changes

All imports must be updated. Replace every old import with the new one:

Old Package (surr-core)	New Package (taal-core)
in.museinc.android.surr_core.recorder.TaalRecorder	com.musediagnostics.taal.TaalRecorder
in.museinc.android.surr_core.recorder.PreFilter	com.musediagnostics.taal.PreFilter
in.museinc.android.surr_core.recorder.TaalRecorderState	com.musediagnostics.taal.core.RecorderState
in.museinc.android.surr_core.player.TaalPlayer	com.musediagnostics.taal.TaalPlayer
in.museinc.android.surr_core.utils.SurrUtils	com.musediagnostics.taal.utils.SurrUtils
in.museinc.android.surr_core.utils.TaalConnectionBroadcastReceiver	com.musediagnostics.taal.utils.TaalConnectionBroadcastReceiver
in.museinc.android.surr_core.exceptions.*	com.musediagnostics.taal.TaalDisconnectedException etc.

3.3 API Changes

Area	Old (surr-core)	New (taal-core)
Pre-amp range	0–10 dB	0–30 dB
Filtered file method	setPreFilteredAudioFilePath()	setFilteredAudioFilePath()
Player filter	No setPreFilter() on TaalPlayer	setPreFilter(PreFilter) added
File output	Single raw .wav	Two files: _raw.wav + _filtered.wav
Screen orientation	Not locked	Portrait only (cannot be overridden)
RecorderState enum	TaalRecorderState	RecorderState
Max recording time	Not specified	300 seconds (5 minutes)

3.4 Manifest Changes

Remove old activity declarations and add the new ones:

```
<!-- REMOVE these old declarations -->
<activity android:name="in.museinc.android.surr_ui.RecorderActivity" ... />
<activity android:name="in.museinc.android.surr_ui.PlayerActivity" ... />

<!-- ADD these new declarations -->
<activity android:name="com.musediagnostics.taal.uikit.TaalRecorderActivity" ... />
<activity android:name="com.musediagnostics.taal.uikit.TaalPlayerActivity" ... />
```

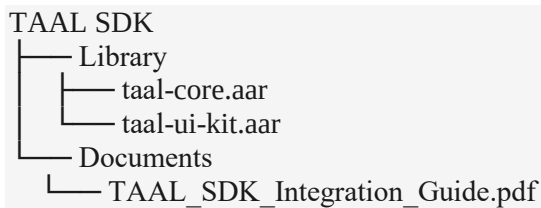
4. Features

- Record audio from the TAAL USB stethoscope device
- Real-time audio filtering (Heart, Lungs, Bowel, Pregnancy, Full Body)
- Pre-amplification control (0–30 dB)
- Real-time waveform visualization during recording
- BPM calculation (Heart filter)
- Dual file output: raw .wav + filtered .wav written simultaneously in real-time
- Pre-built UI for recording and playback (taal-ui-kit)
- Saved recordings library
- USB device connection/disconnection detection
- Portrait-only orientation enforced on all SDK Activities

5. Specifications

Item	Description
Development Environment	Android Studio Hedgehog or later
Minimum SDK API	API 24 (Android 7.0)
Target SDK	API 34
Development Language	Kotlin
Supported Interfaces	USB (OTG-Compatible Android Devices)
Audio Format	WAV, 16-bit PCM, Mono, 44,100 Hz
Pre-amp Range	0–30 dB
Max Recording Time	300 seconds (5 minutes) per session

6. Folder Structure



7. Setup & Installation

The TAAL SDK is built and designed to be used with Android Studio. The following instructions will help you to integrate the TAAL SDK into your application.

Step 1: Add AAR Files

1. Create the app/libs/ directory inside your app module
2. Copy both AAR files into app/libs/:
 - taal-core.aar
 - taal-ui-kit.aar

Step 2: Configure Repositories (settings.gradle.kts)

The UI-Kit uses MPAndroidChart for waveform rendering, hosted on JitPack.

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven { url = uri("https://jitpack.io") } // Required for MPAndroidChart
    }
}
```

Step 3: Add Dependencies (app/build.gradle.kts)

```
android {
    defaultConfig {
        minSdk = 24 // Must be 24 or higher
    }
}

dependencies {
    // TAAL SDK AAR files
    implementation(files("libs/taal-core.aar"))
    implementation(files("libs/taal-ui-kit.aar"))
}
```

```
// taal-core requirements
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")

// taal-ui-kit requirements
implementation("androidx.recyclerview:recyclerview:1.3.2")
implementation("androidx.navigation:navigation-fragment-ktx:2.7.6")
implementation("androidx.navigation:navigation-ui-ktx:2.7.6")
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")
implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.7.0")
implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.7.0")
implementation("com.google.android.material:material:1.11.0")
implementation("com.github.PhilJay:MPAndroidChart:v3.1.0") // Waveform chart
}
```

Step 4: AndroidManifest.xml

Add the required permissions and declare both SDK activities. Use `tools:replace` to resolve manifest merger conflicts.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <!-- Required permissions -->
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <uses-permission android:name="android.permission.USB_PERMISSION" />
    <uses-feature
        android:name="android.hardware.usb.host"
        android:required="false" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.YourApp"
        tools:replace="android:allowBackup,android:label,android:theme,android:icon">

        <!-- Your main activity -->
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- SDK Activities — set exported=false for security -->
        <activity
            android:name="com.musediagnostics.taal.uikit.TaalRecorderActivity"
            android:exported="false"
            tools:replace="android:exported" />
    </application>
</manifest>
```

```
<activity
  android:name="com.musediagnostics.taal.uikit.TaalPlayerActivity"
  android:exported="false"
  tools:replace="android:exported" />

</application>
</manifest>
```

Note: Both SDK activities are locked to portrait orientation. This cannot be overridden.

8. SDK Reference — TaalRecorder

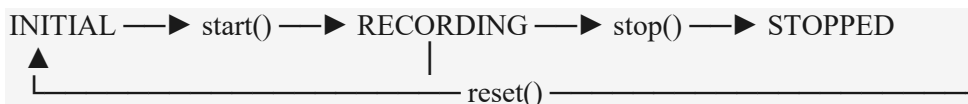
TaalRecorder is the core class for recording audio from the TAAL device.

Import:

```
import com.musediagnostics.taal.TaalRecorder
import com.musediagnostics.taal.PreFilter
import com.musediagnostics.taal.core.RecorderState
```

8.1 State Diagram

Recording from the TAAL device is managed as a state machine. The following shows the lifecycle and states of a TaalRecorder object:



Configuration methods valid in INITIAL state:

```
setRawAudioFilePath(filePath)
setFilteredAudioFilePath(filePath)
setPlayback(playbackFlag)
setRecordingTime(timeInSeconds)
setPreFilter(preFilter)
setPreAmplification(preAmpInDB)
```

8.2 Common Usage

```
val taalRecorder = TaalRecorder(context)

// Required: set raw audio output path
taalRecorder.setRawAudioFilePath("/path/to/recording_raw.wav")

// Recommended: set filtered audio output path (written in real-time)
taalRecorder.setFilteredAudioFilePath("/path/to/recording_filtered.wav")

taalRecorder.setRecordingTime(300) // Max duration in seconds (default 30)
```

```

taalRecorder.setPlayback(false)    // Earpiece monitoring (reserved for future use)
taalRecorder.setPreFilter(PreFilter.HEART)
taalRecorder.setPreAmplification(5) // 0–30 dB

taalRecorder.start() // Throws TaalDisconnectedException if no device connected

// ... recording in progress ...

taalRecorder.stop()
taalRecorder.reset()

```

8.3 Public Methods

Method Name	Valid States	Description
setRawAudioFilePath(path: String)	INITIAL	Sets output path for the raw (unfiltered) WAV file. Must end in .wav.
setFilteredAudioFilePath(path: String)	INITIAL	Sets output path for the filtered WAV file. Written in real-time during recording. Must end in .wav.
setRecordingTime(seconds: Int)	INITIAL	Maximum recording duration. Minimum 1 second. Default 30.
setPlayback(enabled: Boolean)	INITIAL	Reserved for future earpiece monitoring. Currently accepted but has no effect.
setPreFilter(filter: PreFilter)	INITIAL	Sets the acoustic filter preset. Options: HEART, LUNGS, BOWEL, PREGNANCY, FULL_BODY.
setPreAmplification(db: Int)	Any	Sets pre-amplification gain in dB. Range: 0–30 dB. Can be updated during recording.
getState()	Any	Returns current RecorderState: INITIAL, RECORDING, or STOPPED.
setOnInfoListener()	Any	Sets the OnInfoListener for state and progress callbacks.
start()	INITIAL	Starts recording. Throws TaalDisconnectedException if TAAL device is not connected.
stop()	RECORDING	Stops recording and finalizes both output files.
reset()	INITIAL, STOPPED	Resets recorder back to INITIAL state. Clears all paths and settings.

8.4 OnInfoListener — State & Progress Updates

Register to receive state changes and live audio data during recording:

```
taalRecorder.onInfoListener = object : TaalRecorder.OnInfoListener {
    override fun onStateChange(state: RecorderState) {
        when (state) {
            RecorderState.RECORDING -> { /* recording started */ }
            RecorderState.STOPPED -> { /* recording finished */ }
            else -> {}
        }
    }

    override fun onProgressUpdate(
        sampleRate: Int,
        bufferSize: Int,
        timeStamp: Double, // Elapsed recording time in seconds
        data: FloatArray // Filtered float samples, range -1.0 to +1.0
    ) {
        // Called on IO thread — switch to Main thread for any UI updates
    }
}
```

8.5 OnLiveStreamListener — Raw Byte Stream

Receive a raw byte stream of filtered audio (e.g. for network streaming or custom processing):

```
taalRecorder.onLiveStreamListener = object : TaalRecorder.OnLiveStreamListener {
    override fun onNewStream(stream: ByteArray) {
        // 16-bit PCM bytes of the filtered audio — called on IO thread
    }
}
```

8.6 USB Connection Detection

Use `TaalConnectionBroadcastReceiver` to receive live plug/unplug callbacks. Register in `onStart/onResume` and always unregister in `onStop/onDestroy` to prevent memory leaks.

```
import com.musediagnostics.taal.utils.TaalConnectionBroadcastReceiver

val receiver = TaalConnectionBroadcastReceiver(
    object : TaalConnectionBroadcastReceiver.TaalConnectionListener {
        override fun onTaalConnect() {
            // TAAL device plugged in
        }
        override fun onTaalDisconnect() {
            // TAAL device unplugged
        }
    }
)
```

```
// In onStart / onResume
receiver.register(context)

// In onStop / onDestroy — always unregister to prevent memory leaks
receiver.unregister(context)
```

Check connection status on launch (the broadcast receiver misses events that happened before registration):

```
import com.musediagnostics.taal.utils.SurrUtils

val status = SurrUtils.isTaalDeviceConnected(context)
// Returns: CONNECTED, NOT_CONNECTED,
//          DEVICE_DOES_NOT_SUPPORT_OTG, INVALID_TAAL_CONNECTED
```

9. SDK Reference — TaalPlayer

TaalPlayer plays back WAV recordings from the TAAL device with real-time DSP filtering.

Import:

```
import com.musediagnostics.taal.TaalPlayer
import com.musediagnostics.taal.PreFilter
import com.musediagnostics.taal.InvalidFileNameException
import com.musediagnostics.taal.dsp.AudioFilterEngine // Only needed if using setGraphicEQ()
```

9.1 Common Usage

```
val taalPlayer = TaalPlayer(context)
taalPlayer.setDataSource("/path/to/recording.wav") // Throws InvalidFileNameException if not .wav

// Optional: apply filter for raw files only.
// Do NOT call setPreFilter() on _filtered.wav — it is already filtered.
if (!filePath.contains("_filtered")) {
    taalPlayer.setPreFilter(PreFilter.HEART)
}

taalPlayer.setPreAmplification(0f) // 0–30 dB, default 0
taalPlayer.setLooping(false)

taalPlayer.onPlaybackProgress = { timestamp, data ->
    // timestamp = elapsed seconds, data = filtered float samples
}
taalPlayer.onPlaybackComplete = {
    // Playback finished naturally
}

taalPlayer.prepare()
taalPlayer.start()
// ...
```

```
taalPlayer.stop()
taalPlayer.release() // Always call when done to free AudioTrack resources
```

9.2 Public Methods

Method	Description
setDataSource(filePath: String)	Sets the WAV file path. Throws <code>InvalidFileNameException</code> if the file does not exist or is not .wav.
setPreFilter(filter: PreFilter)	Applies an acoustic preset filter during playback. Only call on raw (unfiltered) files.
setPreAmplification(db: Float)	Sets playback amplification in dB. Range: 0–30.
setGraphicEQ(eqState: AudioFilterEngine.GraphicEQState)	Applies a 5-band graphic EQ during playback.
setLooping(loop: Boolean)	Enables continuous looped playback. Default false.
prepare()	Initialises the AudioTrack. Must be called before start().
start()	Starts playback.
stop()	Stops playback.
release()	Releases AudioTrack and all resources. Always call when done.
reset()	Calls release() and clears the data source.

10. SDK Reference — TaalRecorderActivity (UI-Kit)

TaalRecorderActivity provides a complete pre-built recording screen with waveform, filter chips, pre-amp slider, BPM display, and save/discard flow.

Import:

```
import com.musediagnostics.taal.uikit.TaalRecorderActivity
```

10.1 Launch the Recorder

```
private val recorderLauncher = registerForActivityResult(
    ActivityResultContracts.StartActivityForResult()
) { result ->
    if (result.resultCode == RESULT_OK) {
        val filePath = result.data?.getStringExtra(TaalRecorderActivity.RESULT_FILE_PATH)
        // filePath = absolute path to the saved _filtered.wav file
    }
}
```

```
// Launch the recorder
recorderLauncher.launch(
    TaalRecorderActivity.getIntent(
        context      = this,
        preFilter     = "HEART", // HEART | LUNGS | BOWEL | PREGNANCY | FULL_BODY
        preAmplification = 5,    // 0–30 dB, default 5
        recordingTimeSeconds = 300 // Max duration, default 300
    )
)
```

10.2 Activity Result

Result Code	Meaning	Extra
RESULT_OK	User recorded and saved	RESULT_FILE_PATH → absolute path to _filtered.wav
RESULT_CANCELED	User discarded or pressed back	No extra

11. SDK Reference — TaalPlayerActivity (UI-Kit)

TaalPlayerActivity provides a pre-built playback screen with waveform visualisation, play/stop controls, and amplitude adjustment.

Import:

```
import com.musediagnostics.taal.uikit.TaalPlayerActivity
```

11.1 Launch the Player

```
startActivity(
    TaalPlayerActivity.getIntent(
        context = this,
        filePath = "/path/to/recording_filtered.wav"
    )
)

// Pass the _filtered.wav path returned by RESULT_FILE_PATH from TaalRecorderActivity.
// The player detects _filtered in the filename and skips preset filters
// to avoid double-filtering.
```

12. Dual File Output System

Every recording produces two files simultaneously:

File	Suffix	Content	Use
Raw file	_raw.wav	Unfiltered audio from the TAAL device	Archive / re-processing
Filtered file	_filtered.wav	Filter + pre-amp applied in real-time	Playback and waveform display

Why Two Files?

The filtered file is written to disk during recording with zero extra CPU cost — the filter computation already happens for the live waveform. This means the player screen is always instant, with no post-recording processing required.

File Paths (Core SDK)

```
val ts = System.currentTimeMillis()
val rawPath = "${context.filesDir}/recording_${ts}_raw.wav"
val filteredPath = "${context.filesDir}/recording_${ts}_filtered.wav"

taalRecorder.setRawAudioFilePath(rawPath)
taalRecorder.setFilteredAudioFilePath(filteredPath)
```

File Paths (UI-Kit)

Handled automatically. `RESULT_FILE_PATH` always returns the `_filtered.wav` path. Both `_raw.wav` and `_filtered.wav` are saved together to `filesDir/saved/`.

13. Exceptions

Exception	When Thrown
<code>TaalDisconnectedException</code>	<code>TaalRecorder.start()</code> called with no TAAL device connected via USB.
<code>TaalNotAvailableForUseException</code>	TAAL device is in use by another application.
<code>InvalidFileNameException</code>	<code>TaalPlayer.setDataSource()</code> called with a non-.wav path or a file that does not exist.
<code>IllegalArgumentException</code>	<code>setRawAudioFilePath()</code> or <code>setFilteredAudioFilePath()</code> called with a path that does not end in .wav.

DeviceDoesNotSupportOTGConnection	Thrown when the Android device does not support OTG connection.
InvalidTaalConnectedException	Connected TAAL device information does not match MUSE Diagnostics records.
IllegalStateException	Calling setup methods (e.g. setPreFilter()) while recording is in progress, or calling start() without setting rawAudioFilePath first.

14. Miscellaneous Utilities

14.1 SurrUtils

Import: import com.musediagnostics.taal.utils.SurrUtils

Method	Returns	Description
SurrUtils.isTaalDeviceConnected(context)	ConnectionStatus	Checks USB connection. Values: CONNECTED, NOT_CONNECTED, DEVICE_DOES_NOT_SUPPORT_OTG, INVALID_TAAL_CONNECTED
SurrUtils.readSampleRate(filePath)	Int	Returns the sample rate from a WAV file header.
SurrUtils.getFloatBuffer(filePath)	List<Float>	Returns all PCM samples normalised to range -1.0 to +1.0.

14.2 WavCropper (taal-ui-kit)

Import: import com.musediagnostics.taal.uikit.util.WavCropper

Method	Returns	Description
WavCropper.getDurationSeconds(filePath)	Float	Duration of a WAV file in seconds.
WavCropper.getWaveformData(filePath, maxPoints)	FloatArray	Downsampled waveform data for display.
WavCropper.cropWav(inputPath, outputPath, startSeconds, endSeconds)	Boolean	Writes a trimmed WAV file from the given time range.

15. UI-Kit Styling

You can override the following color attributes in your color.xml to customize the Player and Recording Activity UI:

```
<color name="suk_status_bar">#your_color_code</color>
<color name="suk_action_bar">#your_color_code</color>
<color name="suk_action_bar_action_text_icon_active">#your_color_code</color>
<color name="suk_action_bar_action_text_icon_inactive">#your_color_code</color>
<color name="suk_window_background">#your_color_code</color>
<color name="suk_primary_text">#your_color_code</color>
<color name="suk_secondary_text">#your_color_code</color>
<color name="suk_pre_filter_btn_background">#your_color_code</color>
<color name="suk_pre_filter_btn_active">#your_color_code</color>
<color name="suk_pre_filter_btn_inactive">#your_color_code</color>
<color name="suk_pre_amplification_controller_track">#your_color_code</color>
<color name="suk_pre_amplification_controller_thumb">#your_color_code</color>
<color name="suk_graph_grid">#your_color_code</color>
<color name="suk_graph_line">#your_color_code</color>
<color name="suk_player_seek_bar">#your_color_code</color>
<color name="suk_play_pause_start_btn">#your_color_code</color>
<color name="suk_dialog_btn">#your_color_code</color>
```

16. Technical Limitations

1. **AUX headphones conflict:** Android gives higher priority to wired headphone microphones (AUX) than USB. If wired headphones with a built-in microphone are connected, recording will capture from the headphone mic instead of the TAAL device. Use Bluetooth headphones for monitoring — they do not affect USB recording priority.
2. **OTG required:** The TAAL device connects via USB. OTG must be enabled on the Android device. Some devices have OTG always on; others require enabling it in Settings.
3. **Portrait only:** Both TaalRecorderActivity and TaalPlayerActivity are locked to portrait orientation and cannot be changed.
4. **WAV format only:** The SDK only supports .wav files (16-bit PCM, Mono, 44,100 Hz). Passing any other format will throw `InvalidFileNameException`.
5. **Filtered file WAV header:** The `_filtered.wav` file's WAV header size field is updated asynchronously after recording stops. The file is fully playable immediately, but tools that strictly validate header size may read an incorrect duration until the header update completes (typically within 1–2 seconds).

17. Best Practices to Connect TAAL to Android

1. Before using TAAL please make sure you have charged the device properly. Low battery can cause unstable USB connections.
2. TAAL device has a USB-A port for connectivity. For Android devices with USB-C ports, use a USB-C to USB-A OTG adapter. For Android devices with USB-B ports, use an OTG converter with a USB-A(male) to USB-C(male) cable.

3. TAAL device connection requires OTG. Please make sure OTG is enabled on your Android device. On some devices OTG is always enabled; on others you must enable it from the Android Settings screen.
4. Do not use wired headphones with an inbuilt microphone while recording. Android OS gives AUX audio input higher priority than USB, which causes the SDK to record from the headphone mic instead of TAAL. Use Bluetooth headphones instead — they do not take priority over the TAAL device.
5. Always call `release()` on `TaalPlayer` when your `Fragment/Activity` is destroyed to free the `AudioTrack`.
6. Always unregister `TaalConnectionBroadcastReceiver` in `onDestroy` to prevent memory leaks.
7. Check connection status on launch using `SurrUtils.isTaalDeviceConnected(context)` in `onCreate/onResume` — `TaalConnectionBroadcastReceiver` only fires on plug/unplug events and misses devices already connected before registration.
8. Do not call `setPreFilter()` on `_filtered.wav` files — the filter is already baked into the file. Applying it again produces incorrect audio.
9. Keep pre-amplification moderate (5–15 dB) for most clinical environments. Values above 20 dB may saturate the signal in noisy settings.
10. Wrap `taalRecorder.start()` in a try-catch to handle `TaalDisconnectedException` and show users a clear message to connect the TAAL device.

This document and the information contained herein are confidential to and the property of MUSE Diagnostics. Unauthorized access, copying and replication are prohibited. This document must not be copied in whole or part by any means, without the written authorization of MUSE Diagnostics.